דף סיכום בחינה

מזהה סטודנט: [redacted]

מזהה קורס: 20594   שם קורס: מערכות הפעלה

מספר שאלה	ניקוד מירבי	ציון
1	25.00	25.00
2	17.00	17.00
3	13.00	11.00
4	5.00	4.00
5	5.00	5.00
6	5.00	5.00
7	5.00	5.00
8	5.00	5.00
9	5.00	0.00
10	5.00	5.00
11	5.00	5.00
12	5.00	5.00

ציון בחינה סופי : 92.00

הבחינה הבדוקה בעמודים הבאים

[redacted]

[redacted]

מספר התלמיד הנבחן
רשום את כל תשע הספרות

האוניברסיטה הפתוחה

ח' בתמוז תשפ"ב

7 ביולי 2022

מס' שאלון - 525

סמסטר 2022ב

20594 / 4

מס' מועד 89

שאלון בחינת גמר

20594 - מערכות הפעלה

משך בחינה: 3 שעות

בשאלון זה **12** עמודים

מבנה הבחינה:

קראו בעיון לפני שתתחילו בפתרון הבחינה!

א. המבחן מורכב משלושה חלקים.

ב. בחלקים א ו - ב מופיעות שאלות פתוחות. ענו תשובות מלאות, בכתב קריא ובקיצור נמרץ. אין חובה להשתמש בכל השורות המוקצות לצורך התשובות, אך אין לחרוג מהמקום המוקצה.

ג. בחלק ג (שאלות אמריקאיות) עליכם לבחור בכל פעם בתשובה יחידה מבין התשובות המוצעות ולהקיף בעיגול את אות התשובה שבחרתם.

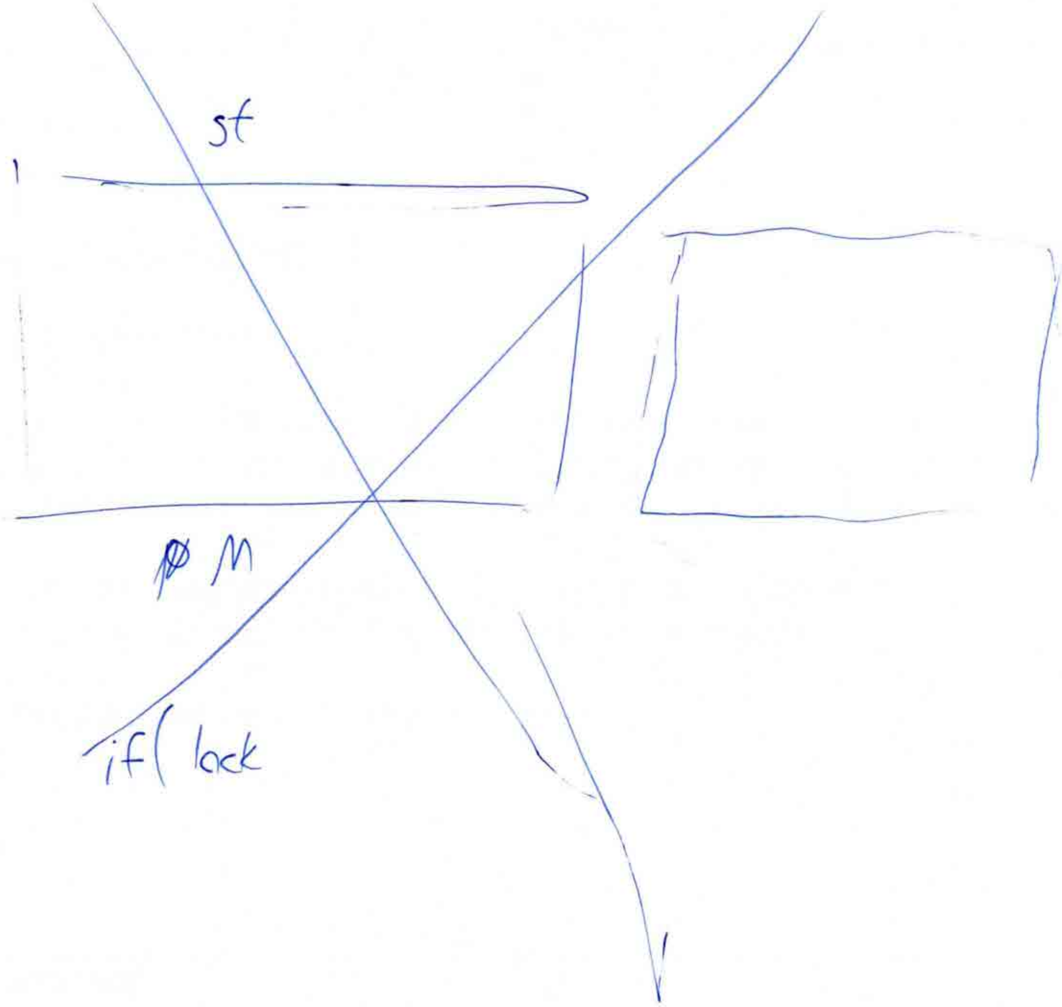
את התשובות לכל השאלות יש לכתוב בשאלון הבחינה.

חומר עזר:
כל חומר עזר אסור בשימוש, פרט למחשבון, שאינו אוצר מידע.

בהצלחה !!!

החזירו
למשגיח את השאלון
וכל עזר אחר שקיבלתם בתוך מחברת התשובות

525 שאלון 89.73.545 M4


st
M
if(lock

הנחיות למבחן

בחינה בעקרונות מערכות הפעלה

קראו בעיון לפני שתתחילו בפתרון הבחינה!

1. בחלקים א׳ ו ב׳ מופיעות שאלות פתוחות. ענו **תשובות מלאות, בכתב קריא ובקיצור נמרץ**. אין חובה להשתמש בכל השורות המוקצות לצורך התשובות, אך **אין לחרוג מהמקום המוקצה.**
2. בחלק ג׳ (שאלות אמריקאיות) עליכם לבחור בכל פעם בתשובה **יחידה הנכונה ביותר** מבין התשובות.
3. כל חומר עזר אסור לשימוש בזמן הבחינה פרט למחשבון פשוט
4. משך הבחינה - שלוש שעות.

בהצלחה!

חלק א (55 נקודות)

שאלה 1 (25 נקודות)

מחשב חדש שנמצא בשלבי פיתוח מבוסס מעבד תומך בשתי פקודות מכונה מיוחדות לתיאום בין תהליכונים(THREADS). הקומפיילר תומך בפקודות המכונה הללו באמצעות מקרואים מיוחדים שמתורגמים לפקודות המכונה.

פקודת מכונה אחת מפעילים בעזרת המקרו m שהוא שקול לאופרטור * על מצביעים, כלומר v=m(p) שקול לטעינת הערך ששמור בכתובת p לתוך המשתנה v.

המקרו השני נקרא st והוא מיוחד יותר. הביטוי $r = st(v, p)$ מנסה לכתוב את הערך v לכתובת p בזיכרון.

הכתיבה מצליחה ומוחזר הערך 1, אם ורק אם התהליכון שמבצע את$st(v, p)$ ביצע לפני כן m(p) על הכתובת p וזו הפעולה האחרונה שבוצעה על הכתובת p ואף תהליכון אחר לא כתב ערך לכתובת p מאז שהתהליכון הזה ביצע m(p) .

בכל מקרה אחר, כלומר אם התהליכון לא ביצע את m או אם תהליכון אחר כתב לכתובת p מאז שהוא ביצע את m (בין אם בעזרת st ובין אם בעזרת השמה בכל דרך), הכתיבה נכשלת (הערך v לא נכתב לכתובת p) והמקרו מחזיר את הערך 0 . במילים אחרות, הכתיבה מצליחה רק אם הערך שכתוב בזיכרון בעת ניסיון הכתיבה הוא אותו הערך שהתהליכון בעצמו קרא בעזרת ה m ואף תהליכון אחר לא כתב לתוכו מאז.

(המשך השאלה בעמוד הבא)

525 שאלון 89.73.545 M4

טיוט,

```
lock(p)      while(*p==1);
             while if (*p==0)
while(*p=0)  if(str(1,p)!=1)
  m(p)=0       m(p)=0
  str(1,p)     while(*p==1);

             while(!str(1,p)).
```

```
lock(p)        lock(p)
m              m
st             st
M              M

M                      p=0
—
                lock=0
—               p
```

```
lock(p)
  while(*p==1);
  while(*p==0){
      m(p)=1
      if(str(1,t)!=1)
          return;
      while(*p==1);
  }
```

צריך להשתמש בפקודות כמו
שהם הוגדרו בשאלה. אפשר להשתמש
בפקודות רגילות של שפת תכנות.
אסור להשתמש במספרים ושמות
מנכרין מוקנים אחרים.

הערה: הנחתי שמה שהכוונה ב m(p) היא שאתחל ערך באוף p.
וכן השתמשתי בו בהגדרה: m(p)=1 כלומר אכסנתי 1 באוף p.

א. **(15 נק')** ממשו שגרות lock(p) ו unlock(p) למניעה הדדית, כאשר p הוא מצביע לשלם שמייצג את המנעול. המימוש צריך להשתמש במקרואים(MACROS) המתוארים בשאלה.

```
lock (p) {
    while (*p==1) {busy waiting}
    while (*p==0) {
        m(p)=1
            if (str(1,p)==1)
                exit; (יצא מהלולאה)
            else
                while (*p==1) {busy waiting}
    }
unlock(p) {
    *p=0 }
```

בשלב זה התהליך צריך לבדוק אם הוא התהליך היחיד שתפס את המנעול או שיש נוסף. הבדיקה נעשה באמצעות str

ב. **(10 נק')** מוגדר מבנה נתונים של רשימה מקושרת של מבנים מהטיפוס:

struct item { int data; struct item *next }

עם מצביע struct item *head לראש הרשימה. ממשו שגרה insert(int d) שמכניסה איבר חדש לרשימה שערך השדה data בו הוא d, וזאת תוך שימוש במקרואים m ו st. אסור להשתמש במנעולים ואסור להגדיר משתנים נוספים (פרט למשתנים אוטומטיים בשגרה).

השגרה צריכה לפעול נכון גם בסביבה שבה יש עוד תהליכונים שמנסים לפעול על הרשימה במקביל.

```
insert item(int d) {
    produce item with data=d;
    while (*p==1) {busy waiting}
    while (*p==0) {
        m(p)=1
        if(str(1,p)==1) {exit;} (יצא מהלולאה)
        else {while (*p==1) {busy waiting}}
    }
    item.next = head
    head = item
    *p=0
}
```

הקטע הקריטי - הוספת ה item לרשימה

25
(1)

525 שאלון 89.73.545 M4

נא לא לכתוב בשוליים

שאלה 2 (17 נקודות)

מערכת מחשב מסוימת משתמשת בדפים בגודל 4KB ודיסק **קשיח** לדפדוף שקצב העברת הנתונים שלו הוא 10MByte/sec וזמן חיפוש(SEEK) ממוצע בו הוא 10ms(10 אלפיות שנייה). כל המרחב הווירטואלי של כל תהליך שמור בדיסק בצורה רציפה. הניחו שבכל שגיאת דף(PAGE FAULT) קיימות 2 אפשרויות בלבד: המערכת תביא דף בודד או 10 דפים רצופים(כמו שנאמר הם שמורים בדיסק ברצף).

הערה: בחישוב לא צריך לקחת בחשבון את הזמנים של פעולות טיפול אחרות בשגיאת דף (PAGE FAULT) שלגביהם אין נתונים בשאלה.

א. **(4 נק')**

האם מנגנון הבאת דפים מראש(PREPAGING) עשוי להאט תכניות לעומת המערכת דפדוף לפי דרישה(ON DEMAND PAGING)? נמקו. אם עניתם שכן, מהי ההאטה המרבית(במה ואיך היא תלויה)? תכתבו תיאור מילולי בלי חישובים.

תשובה והסבר:

מנגנון הבאת דפים מראש יכול להאט את התוכנית משום שמנגנון prepaging מביא בכל פעם דף בודד, שאז כל דף מתבצע seek של חיפוש. אם הקריאה היא קריאה רציפה, אז מנגנון ה prepaging היה מתבצע seek של כל דף, לעומת מנגנון ה on demand paging שהיה מתבצע רק seek של הבאת הדפים! (נמוך שההאטה זו תהיה רק כשהגישה היא רציפה, כשהגישה היא אקראית אז התקורה המתקבלת מחיפוש זמן הבאת דף תהיה בשני המנגנונים)

ב. **(7 נק')**

תארו מנגנון פשוט לקבלת החלטה ע"י מערכת הפעלה בזמן טיפול בשגיאת דף האם להביא דף בודד או 10 דפים. תארו את מבני הנתונים הדרושים למנגנות ואיזה גורם מעדכן את הנתונים האלה. המנגנון צריך לצרוך לא יותר ממספר קטן וקבוע של בתים בזיכרון לכל תהליך ולא יותר ממספר קטן וקבוע של פעולות חישוב בכל שגיאת דף. המנגנון חייב לקבל בד"כ החלטה נכונה במצבים שבם מתבצעות גישות אקראיות לזיכרון ראשי(למשל גישות לטבלת HASH) ובמצבים שבהם מתבצעות גישה רציפה לזיכרון (למשל חיפוש לינארי בטבלה גדולה). **מותר למנגנון לקבל החלטות שגויות במקרים אחרים.**

תשובה והסבר:

המנגנון יהיה תור של גודל התור יהיה קבוע ונתון לפי המערכת (נסמן את הגודל ב n). בכל שגיאת דף נכניס את מספר הדף שמבקשים לסוף התור, ונוציא מראש התור את מספר הדף שהיה שם (תור הדף שבקשו לפני n page faults). בכל שגיאת דף נבדוק האם התור מכיל סדרה רציפה של מספרים, לדוגמא: 5,6,7,8 (5 ברצף הדומה) או סדרה של מספרים שאינה רציפה לדוגמא: 1,7,3,12,4,77. אם התור מכיל סדרה רציפה של מספרים אז נביא 10 דפים רצופים (משום שהסדרה הרציפה מראה לנו שהגישה היא גישה רציפה לזיכרון) ואם הסדרה אינה רציפה נביא רק דף בודד (משום שהגישה היא אקראית).

(המשך השאלה בעמוד הבא)

ג. **(6 נק')**

האם המנגנון שתיארתם בסעיף ~~א~~ ב' פועל בצורה נכונה כאשר התכנית מחשבת מכפלה פנימית של שני ווקטורים ארוכים? (כלומר עבור הלולאה ;for (i=0; i<N; i++) s+=A[i] * B[i]).
הוכיחו שכן או הסבירו למה לא ותארו כיצד ניתן לשפר את המנגנון כך שיפעל נכון גם במקרה הזה.

17
(2)

תשובה והסבר:

המנגנון לא יעבוד בצורה אופטימלית. נניח שוקטור A נמצא בדפים 10, 11, 12, 13 ומנגנון B נמצא בדפים: 30, 31, 32, 33. במקרה זה האור יהיה: 10, 30, 11, 31 [illegible] לא נשמור על סדרה רציפה ולכן יובא בכל פעם רק דף בודד. אם היינו מביאים 10 דפים רציפים מוקטור A, ו-10 דפים רציפים מוקטור B היינו חוסכים הרבה page fault-ים וחוסכים את ה-[illegible] שבזבזנו בכל הבאת דף כשהיינו מחפשים את מיקום הדף.

אפשר לנסות לשפר את המנגנון ע"י כך שלא נבדק רק אם הסדרה רציפה או לא, אלא ננסה למצוא את סדרות עולות רציפות. למשל בסדרה הבאה: 10, 30, 11, 31, 12, 32, 13 המנגנון יזהה את הסדרה העולה 1, 2, 3 ויביא 10 דפים רצופים ולא רק דף בודד.

המשך הבחינה בעמוד הבא

525 שאלון 89.73.545 M4

נא לא לכתוב בשוליים

שאלה 3 (13 נקודות)

כידוע, אחת הבעיות הפוגעות בביצועי מערכת הקבצים היא קבצים שאינם שמורים באופן רציף על הדיסק הקשיח.

א. **(3 נק')**

נציע שיטה חדשה להקצאת מקום לקובץ - "שיטת הרווחים". השיטה תשמש לצמצום ממדי הבעיה ע"י השארת מקום ריק (רווח) בגודל קבוע לאחר כל קובץ, כך שאם הקובץ צריך לגדול, הוא יכול לגדול באופן רציף. המקום השמור לאחר כל קובץ מיועד לקובץ עצמו, אבל מותר להשתמש במקום זה עבור הקצאה לקבצים אחרים אם אין ברירה אחרת (כלומר רק במקרה שאין יותר מקום פנוי על הדיסק חוץ ממקומות השמורים לגדילה של קבצים).

תנו דוגמה פשוטה למקרה בו, כשמשתמשים ב"שיטת הרווחים" אחד הקבצים אינו שמור באופן רציף.

תשובה והסבר:

נניח שהדיסק המלא ולא נותר עוד מקום בדיסק מלבד המקומות השמורים שנועדו של הקבצים. ובמצב זה הגיע קובץ נוסף, F_2. ומחוסר ברירה, F_2 נשמר במקום ששמור לגדילה של F_1. נניח שלאחר מכן F_1 רוצה לגדול - אך אין לו מקום כי F_2 תפס לו את המקום שהיה אמור להיות שמור לו. במקרה זה F_1 לא ישמר באופן רציף על הדיסק.

ב. **(5 נק')**

הועלתה הצעה הבאה ("שיטת החצי"): הקובץ הראשון ישמר בתחילת הדיסק. בהמשך, בכל פעם שנוצר קובץ חדש, נאתר את השטח הפנוי הרצוף הגדול ביותר על הדיסק, ונשמור את הקובץ החדש החל מאמצע שטח זה.

הסבירו מה היתרון לשמירת קובץ חדש באמצע הקטע הפנוי הרצוף הגדול ביותר ולא בתחילתו?

האם שיטה זו תמנע לחלוטין מצב בו קובץ שמור בכמה חלקים נפרדים (כלומר, האם כל הקבצים יהיו שמורים ברצף)? נמקו.

תשובה והסבר:

מכיוון ששומרים כל קובץ באמצע השטח הפנוי הרציף הגדול ביותר, הדבר מותיר מקום לקבצים לגדול. שיטה זו אכן תגרום ליותר קבצים להיות שמורים באופן רציף, אך לא תמנע את הדבר לחלוטין. יתכן לדוגמא שבשלב מסוים הדיסק מאוד ימלא והשטח שנותר ל F_1 על מנת לגדול אינו מספק והוא צריך שטח נוסף - שלא יהיה רציף. או אפשרות נוספת שקובץ מגיע ומחפש את הקטע הפנוי הרציף הגדול ביותר - אך קטע זה אינו מספיק לו מכיוון שהוא צריך שטח יותר גדול. ואז לא תהיה ברירה אלא לשמור את הקובץ בצורה לא רציפה.

(המשך השאלה בעמוד הבא)

ג. **(5 נק')**

נניח שאנו משתמשים ב"שיטת החצי" כדי למקם קבצים חדשים. מבין השיטות למימוש קבצים שלמדנו, איזו המתאימה ביותר לצרכנו? השיטות שלמדנו הן מימוש קבצים באמצעות הקצאה רציפה (contiguous allocation), מימוש קבצים באמצעות הקצאה בשיטת רשימה מקושרת (linked list allocation), מימוש קבצים ברשימה מקושרת באמצעות אינדקס (מבוססת i-nodes)?. נמקו.

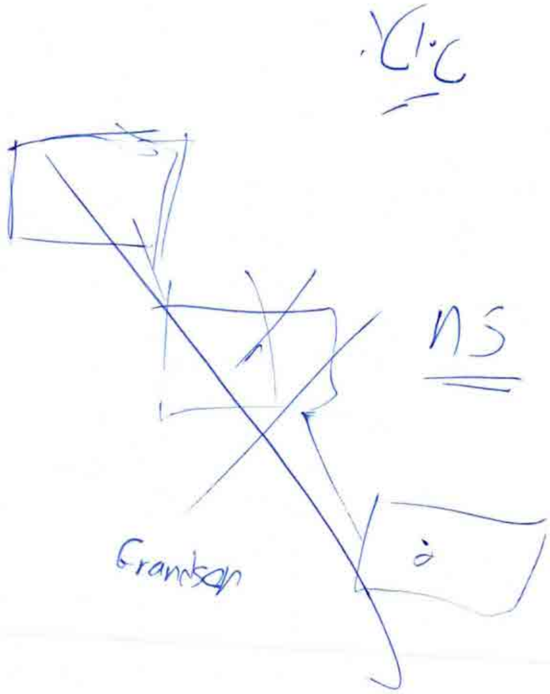
11
(3)

תשובה והסבר:

השיטה המתאימה היא ~~שיטה~~ מימוש באמצעות הקצאה בשיטת רשימה מקושרת.
כיוון שבזמן שרוב הקבצים ישמרו בצורה רציפה אז הרשימה של כל
קובץ תהיה יחסית קצרה, ולכן התקורה של גישה אקראית לא תהיה גבוהה,
וכן בצורה זו יהיה נוח ~~[illegible]~~ לחפש אחר מקום לקובץ חדש שנוצר.

באקראי
בד"כ הגישה איטית

המשך הבחינה בעמוד הבא


NS
Grandson

חלק ב (25 נקודות)

לפניכם חמש שאלות. יש לענות על כולן.

5 נקודות לשאלה ; 25 נקודות לחלק זה.

שאלה 4 (5 נקודות)

הסבירו מה תהיה המשמעות אם במערכת XV6 שתומכת בקונטיינרים לא בוצע בכלל כתיבת "+cpu" לקובץ cgroup.subtree_control : קונטיינר לא ייווצר, ייווצר, אך לא יתפקד בכלל כי לא יקבל את זמן CPU או ייווצר, אך לא יתפקד כמצופה ?

4 (4)

תשובה והסבר :

אם לא נכתוב cpu+ בקובץ ~~[illegible]~~ הקונטיינר
יווצר, (ויתפקד) אך לא כל הקונטרולים יופיעו. חלק מהקונטרולים כמו סטים ל cpu
מופיעים רק אחרי שכותבים cpu+ בקובץ.
(punch עושה זאת באופן אוטומטי ביצירת קונטיינר)

שאלה 5 (5 נקודות)

במערכת XV6 קיימת קריאת מערכת KILL (PID) שמאפשרת להרוג את התהליך ע"פ PID שלו.
פונקציה getppid() ~~מחזיקה~~ מחזירה את ה PID של האב(FATHER), ל PROCESS עם PID=1 , היא מחזירה 0.

מה יהיה הפלט אחרי הרצת הפקודות הבאות במערכת XV6 שתומכת בקונטיינרים?

```
unshare(PID_NS);
int pid = fork();
if(pid == 0) {
  pid = fork();
  if(pid == 0) {
   sleep(999); // long sleep
   kill(getppid());
   printf(stdout, "GrandSon PID=%d \n", getpid());
    }
   else {
     sleep(9999); // very  long sleep
printf(stdout, "Son PID=%d \n", getpid());
     }
   }
  else {
   sleep(99999); //very very sleep enough for creating and running all son processes
   printf(stdout, "Father still alive \n");
  }
```


525 שאלון 89.73.545 M4

[illegible]

ניתן להניח שכל קריאות מערכת מצליחות ושלא נוצרו תהליכים נוספים במערכת מרגע הרצת הקוד(חוץ מאלה שהקוד יוצר) ושמערכת מקצה PID עוקב כל פעם שנוצר תהליך חדש.

תשובה והסבר :

נקרא לתהליכים בשמות: סבא, אבא ובן. האבא והבן נמצאים בתוך PID namespace ולכן ה PID של האבא הוא 1, ושל הבן הוא 2. הבן קורא ל kill(getppid) שאומר להרוג את האב, והאב נהרג. אך, ב xv6 כאשר בתוך namespace התהליך בעל ה PID=1 נהרג, כל הבנים והתהליכים בתוך הקונטיינר גם נהרגים, ולכן הבן לא מספיק לכתוב שום דבר. הסבא כותב: father still alive.

לסיכום, הפלט יהיה: Father still alive.

5 (5)

שאלה 6 (5 נקודות)

מה יודפס בעקבות הרצת קטע קוד הבא בתוך מערכת xv6 שתומכת בקונטיינרים, ניתן להניח שכל קריאות מערכת מצליחות :

```
int pid = fork();
if (pid == 0) {
  mkdir ("dirS");
  mount ("internal_fs_a","dirS",0); //mount a preformatted internal_fs_a
  int fd = open ("dirS/file.txt",O_CREATE|O_RDWR);
  write (fd ,"123456789\n",10);
  close (fd);
 }
if (pid > 0) {
sleep (10000); // wait for child to finish
unshare (MOUNT_NS);
 }
sleep (10000); // both processes will sleep
if( pid == 0) {
  if (open ("dirS/file.txt",O_RDONLY) < 0) printf (stderr, "son failed to open\n");
  else printf (stderr, "son succeeded\n");
 }
If (pid>0) {
  if (open ("dirS/file.txt",O_RDONLY) < 0) printf (stderr, "father failed to open\n");
  else printf (stderr, "father succeeded\n");
 }
```

תשובה והסבר :

5 (6)

נשים לב, שהילד יוצר את dirS ועושה mount לפני שהאב קורא ל unshare. ולכן dirS וכל הקבצים שבתוכו קיימים גם אצל האב גם שם. ולכן הפלט יהיה: son succeeded ! father succeeded

(או להפך, תלוי בהחלטת המתזמן)

המשך הבחינה בעמוד הבא

שאלה 7 (5 נקודות)

לדיסק בעל 40 גלילים(CYLINDERS), כאשר הזרוע נמצאת בגליל 13 (בכיוון תנועה ושירות בקשות כלפי מעלה- הגליל בעל המספר הגבוה ביותר), מגיעה סדרת בקשות קלט/פלט לגלילים הבאים (משמאל לימין):

22, 11, 36, 15, 39, 1

איזה מאלגוריתמי תזמון הדיסק הבאים SSF - החיפוש הקצר קודם , SCAN - אלגוריתם המעלית
ו C-SCAN - סריקה מחזורית בכיוון למעלה בלבד יתבצע תוך השגת מינימום מרחק חיפוש כולל? הראו את החישובים בבירור.

5
(7)

חישוב והסבר:

סדר טיפול הבקשות ב SSF יהיה: 15, 11, 1, 22, 36, 39 ולכן המרחק הכולל הוא:
54 = 2+4+10+21+14+3 . סדר הטיפול ב SCAN: 15,22,36,39,11,1
המרחק הכולל הוא: 64 = 2+7+14+3+28+10 . סדר הטיפול ב C-SCAN הוא:
15,22,36,39,1,11 המרחק הכולל הוא: 74 = 2+7+14+3+38+10
לכן סה"כ קיבלנו שהמרחק הכולל המינימלי יהיה באלגוריתם תזמון SSF.

שאלה 8 (5 נקודות)

5
(8)

איזו תכונה חשובה נדרשת למעבד בשביל שיהיה ניתן לממש עליו את ביצועי הווירטואליזציה בשיטת Trap-and-Simulate?

יש הוראות רגישות - כלומר הוראות שמתנהגות שונה במצב משתמש ובמצב מיוחס
ויש הוראות מיוחסות - כלומר הוראות שקוראות ל Trap, לכן צריך שיהיה
כדי להשתמש ב Trap and Simulate צריך שקבוצת ההוראות
הרגישות תהיה את קבוצת של ההוראות המיוחסות. כלומר שכל הוראה
רגישה תקרא ל Trap ~~[illegible]~~ מכיוון שבצורה כזו ההיפרוויזור
יכול לקרוא פיקוד ~~[illegible]~~ ולחקות את איך שהדבר היה מתבצע
על החומרה האמיתית.

המשך הבחינה בעמוד הבא

$16843018\,kB$

$\frac{2^{n}}{2^{2}} = 2^{8}$

$(2^{24} + 2^{16} + 2^{8} + 10)\,kB$

$16\,GB +$

$2^{8^{3}} = 2^{24}\,kB = 2^{34} = 2^{4} \cdot 2^{30} = 16\,GB$

$10 \cdot 1\,kB + 256\,kB + 256 \cdot 256\,kB + 256 \cdot 256 \cdot 256\,k$

חלק ג (20 נקודות)

לפניכם 4 שאלות. יש לענות על כולן.

(5 נקודות לשאלה ; 20 נקודות לחלק זה).

שאלה 9 (5 נקודות)

אחד ההבדלים בין פונקציות ספריה בשפת C (כגון printf) לבין קריאות מערכת (system calls) ב- Linux הוא :

א. רק למנהל המערכת (super user) מותר להשתמש בקריאות מערכת, בפונקציות ספריה יכול להשתמש כל אחד

ב. **כל** קריאות המערכת הן חלק מפונקציות הספרייה של שפת C

ג. **כל** פונקציות הספרייה ממומשות בעזרת קריאות מערכת

ד. כל התשובות הקודמות אינן נכונות

0 (9)

תשובה ג

שאלה 10 (5 נקודות)

מערכת הקבצים של מערכת הפעלה מסוימת משתמשת בשיטת ה I-node.

- גודל הבלוק במערכת הקבצים הוא 1 KBytes
- כתובת הבלוק היא 4 בתים (bytes)
- 10 שדות של ה I-node יכולים להחזיק ישירות כתובת הבלוק בדיסק
- שדה נוסף אחד נועד להחזיק כתובת של ה single indirect block
- עוד שדה נוסף אחד נועד להחזיק כתובת של ה double indirect block
- ועוד שדה נוסף אחד נועד להחזיק כתובת של ה triple indirect block

כל בלוק מכיל 2^8 כתובות.
סך גודל הקובץ המקסימלי הוא:
$\approx (2^7)^3 2^{10} + (2^8)^2 \cdot 2^{10} + 2^8 \cdot 2^{10} + 10 \cdot 2^{10}$
$\approx 2^{34} = 16GB$

5 (10)

חשבו מהו הגודל המקסימלי של קובץ במערכת הקבצים הזו?

הערה : לצורך החישוב 1 Kbyte = 1024 Byte , 1 GByte = 1024 * 1024 Kbyte

א. 15 GBytes

ב. 15.5 GBytes

ג. 16 GBytes

ד. 16.5 GBytes

המשך הבחינה בעמוד הבא

525 שאלון 89.73.545 M4

$\frac{2^{20}}{2^{12}} = 2^8 \cdot 4 =$

שאלה 11 (5 נקודות)

מהו החיסרון המובהק של שיטת ה- rendezvous לעומת ה- mailbox ב- message passing?

א. שיטת ה-rendezvous קשה יותר למימוש במערכת הפעלה.

ב. שיטת ה-rendezvous פחות גמישה מכיוון שהתהליכים (השולח והנמען) הופכים להיות מסונכרנים (תלויים זה בזה).

ג. בשיטת ה-rendezvous לא ניתן לממש מניעה הדדית בגישה לקטעים קריטיים (critical sections).

ד. שיטת ה-rendezvous גורמת לעומס קיצוני על מנגנון התזמון(SHEDULING)

שאלה 12 (5 נקודות)

נתון כי במערכת מסוימת המשתמשת בשיטת הדפדוף (paging) לצורך ניהול הזיכרון :

- גודל הדף הוא 4 K.
- הכתובת הוירטואלית הנה בת 20 סיביות (bits).
- גודל הזיכרון הפיזי הוא 1024 KB =1 MB.
- גודל שורה בטבלת הדפים (page table entry) הוא 4 בתים (bytes).

אם טבלת הדפים הנה טבלת הדפים המהופכת (inverted page table) מהו גודל הטבלה?

א. 2560 בתים.

ב. 512 בתים.

ג. 2048 בתים.

ד. 1024 בתים.

בהצלחה!

נא לא לכתוב בשולי [illegible]